Atelier 3 :

partie 1 => configuration et création du projet

1 => à quoi sert la commande git init ?

- Elle permet d'initialiser un dépôt Git local dans un dossier.

2 => Quel est le rôle des commandes git config ?

- Elles permettent de configurer Git, notamment le nom et l'adresse e-mail de l'utilisateur.

3 => Que signifie l'expression « working directory » ?

- c'est le répertoire de travail dans lequel on crée et modifie les fichiers du projet.

partie 2 => suivi des fichiers et premier commit

1 => Quelle est la différence entre un fichier tracked et untracked ?

- un fichier tracked est suivi par Git, tandis qu'un fichier untracked n'est pas encore suivi par Git.

2 => peut-on ajouter plusieurs fichiers avec une seule commande git add ?

- oui, par exemple :

git add index.html style.css README.md

3 => à quoi sert le fichier README.md

- il sert à présenter le projet, son objectif et les instructions d'utilisation.

partie 3 => modifier un fichier

1 => que montre git diff ?

- Elle affiche les modifications effectuées dans les fichiers qui ne sont pas encore ajoutées à la zone de staging.

2 => Quelle est la différence entre git diff et git diff --staged ?

git diff affiche les modifications non indexées, tandis que git diff --staged affiche les modifications déjà ajoutées à la zone de staging.

3 => pourquoi créer plusieurs commits au lieu d'un seul ?

- pour sauvgarder les différentes étapes du projet, suivre les modification et faciliter le retour à une version précédente.

partie 4 => gérer plusieurs fichiers

partie 5 => Bilan et vérification.

Commande	rôle
git init	initialiser un dépôt Git local.
git status	Afficher l'état des fichier du dépôt
git add	Ajouter les fichiers à la zone de staging.
git commit	enregistrer les modifications dans l'historique
git log	afficher l'historique des commits.
git diff	afficher les différences entre les modifications.

l'examen de fin module

partie 3 ⇒

- Quelle commande permet de voir l'historique des commits.

```
git log
```

- comment créer une nouvelle branche feature-xyz à partir de main

```
git switch main
git switch -c feature-xyz
```

- Quelle commande permet de fusionner feature-xyz dans main

```
git switch main
git merge feature-xyz
```

- comment annuler les modifications locales avant de commit

```
git restore.
```

- comment vérifier l'état actuel du dépôt.

```
git status
```

partie 2 => gestion de version avec git

- créer le dossier et initialiser le dépôt Git.

```
mkdir depot_local
cd depot_local
git init
```

- lier le dépôt local à un dépôt distant sur Github.

```
git remote add origin
    https://github.com/USER/NAME/
        Repository.git
```

- Ajouter trois fichiers et effectuer le premier commit

```
touch produit.html style.css script.js
git add produit.html style.css script.js
git commit -m "premier commit"
```

- créer la branche gestion produits et basculer dessus.

```
git switch -c "gestion produits"
```

modifier les fichiers, commiter, revenir sur master et fusionner.

```
git add .
git commit -m
git switch master
git merge
```

- Afficher le log sur une seule ligne, afficher les information du dépôt distant et envoyer le travail.

```
git log --oneline
git remote -v
git push -u origin master.
```

- effacer l'historique des commits et supprimer les modifications des fichiers.

```
git switch --orphan nouvelle_branche
git add .
git commit -m
git branch -D master
git branch -m master.
```